Pipes, Functions, and Iteration

HES 505 Fall 2025: Session 6

Matt Williamson

What is literate programming?

Putting the fun in `function`

- What sorts of things do you often copy and paste?
- Has anything gone wrong?
- Is it fun to read?

Anatomy of a function

- Arguments: What does the function need to know to run?
- Parameters: The values you supply to the arguments
- Body: The actual steps in the functions recipe
- Return: What does the function output?

Anatomy of a function

```
find_largest <- function(df, val, size){
  read data
  group by grp
  calculate sum(val) in groups
  filter top size obs
  return filtered df
}
```

Anatomy of a function

```r
find_largest <- function(df, grp, val, size){
  df %>% #read data
  group_by({{ grp }}) %>% #group by grp
  summarize(total = sum({{ val }}, na.rm=TRUE) %>%  #calculate sum in group
  slice_max(., n = size) #filter top size obs
  #return filtered df
}
```

Helpful Hints: Functions

- Use `print("some helpful message")` to check intermediate steps in your function
- The `{{ }}` allows you to pass variables (not values)
- use explicit `return()` to return more than the last step of the function

Documenting Functions

- What does this do?
- What arguments does it accept and what type should those arguments be?
- What does it return?
- See `roxygen2`

Readable scripts: Pipes

- *Pipes* are helpful when the output of a function *depends* on something before it
- Prevents intermediate processing steps from flooding your environment
- More readable then nested function calls
- Thinking about your functions…

Helpful Hints: Pipes

- `|>` and `%>%` pass *outputs* of the left-hand as *inputs* to the right-hand
- Default is first argument of right-hand
- `.` is the placeholder for `%>%` and `_` is the placeholder for `|>`
- `_` is much simpler and has less functionality

Readable Documents: Iteration

- Doing the same thing over and over
- `for` loops are the classic use

```
long_vector_of_inputs
long_vector_of_outputs
for (i in 1:length(long_vector_of_inputs)){
  long_vector_of_outputs[i] <- do_something(arg1 = long_vector_of_inputs[i]
}
```

Readable Documents: iteration

- `map` and `map_`: Do this function for each element of a vector or list
- Default returns a `list`; suffixes can change this (e.g., `map_dfr`)

```
long_list_of_outputs <- map(long_vector_of_inputs,
                            do_something)
```

Readable Documents: iteration

- Also works with 1-time functions

```
long_list_of_outputs <- map(long_vector_of_inputs,
                            function(x) do_something(x) %>%
                              do_something_else())

long_list_of_outputs <- map(long_vector_of_inputs,
                            \(x) do_something(x) %>%
                              do_something_else())
```

A Note on serial vs. parallel processing

- purrr still does things seriously
- furrr allows things to run in parallel
- Works when one step isn't dependent on the other

Bringing it all together: Project Management

Directory structure

The `data` folder

The `scripts` folder

The `docs` folder

Avoiding pitfalls with `git`

- Always pull before you start working on anything new
- Avoid committing large files
- Using `.gitignore`